

Marcar como hecha

Angular JS

De lo que se trata, en principio, es de hacer cuantas menos llamadas al servidor mejor. Hablamos de las SPA; simple page application. Dentro de la misma página se va interactuando. Más adelante veremos que a la hora de cargar cosas del servidor, se carga únicamente los fragmentos de página necesarios.

Esto agilizó mucho la visualización de las páginas interactivas, y las dinámicas (latencia)

El framework Angular, antes de comenzar a tener versiones con números, comenzó por denominarse AngularJS.

Era una librería que se descargaba dentro de la cabecera (head)

La versión en el momento de escribir esto

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
```

Dentro de la etiqueta de inicio html, se escribe ng-app.

luego es posible trabajar con el típico binding de angular que va escribiendo el texto en tiempo real en la medida que se escribe en un campo de texto

```
<div>
  <label>Nombre:</label>
  <input type="text" ng-model="yourName" placeholder="Ingrese nombre">
  <h1>Hola {{yourName}}</h1>
</div>
```

el htm completo:

```
<!DOCTYPE html>
<!-- se escribe en la etiqueta html ng-app -->
<html lang="en" ng-app>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>primero</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
  <div>
    <label>Nombre:</label>
    <input type="text" ng-model="yourName" placeholder="Ingrese nombre">
    <h1>Hola {{yourName}}</h1>
  </div>
</body>
</html>
```

Añadir variables de Angular

Allí donde se declara que se trata de una aplicación angular (en etiqueta html), es posible incluís una o más variables.

```
<html lang="en" ng-app ng-init="yourName='mundo'">
```

En este caso lo utilizamos de manera que la aplicación antes de escribir el nombre permita que haya algo en el campo.

El resto es igual, allí donde se escribía el nombre en el script anterior, en el nuevo tendrá un valor por defecto:

```
<!DOCTYPE html>
<!-- se escribe en la etiqueta html ng-app -->
<!-- 2 añadimos una variable por defecto -->
<html lang="en" ng-app ng-init="yourName='mundo'">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>segundo</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
  <div>
    <label>Nombre:</label>
    <input type="text" ng-model="yourName" placeholder="Enter a name here">
    <h1>Hola {{yourName}}</h1>
  </div>
</body>
</html>
```

Trabajando con variable en AnglarJS

Una vez que definimos variables en la etiqueta html, es posible trabajar con ellas con operadores simples

```
<!DOCTYPE html>
<!-- se escribe en la etiqueta html ng-app -->
<!-- añadimos una variable por defecto -->

<!-- 3 definimos variable con las que trabajaremos -->
<html lang="en" ng-app ng-init="velocidad_maxima=90; velocidad_comprobada=105">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>tercero</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
  <div>
    <h1>Velocidades</h1>
    <h3>Velocidad máxima permitida {{velocidad_maxima}}</h3>
    <h3>Velocidad constatada {{velocidad_comprobada}}</h3>
    <hr>
    <h1>Rebasamiento de velocidad</h1>
    <h3>el vehículo circulaba {{velocidad_comprobada - velocidad_maxima}} por sobre la velocidad permitida</h3>
  </div>
</body>
</html>
```

Creamos y trabajamos con variables interactivamente

En este, similar al anterior, las variables no se declaran en init, sino que son inicializadas al ingresarlas

```
<!DOCTYPE html>
<!-- se escribe en la etiqueta html ng-app -->
<!-- añadimos una variable por defecto -->

<!-- 4 quitamos las variables que las crearemos interactivas -->
<html lang="en" ng-app >
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>cuarto</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
  <div>
    <h1>Velocidades</h1>
    <input type="text" ng-model="velocidad_maxima" placeholder="Ingrese velocidad máxima">
    <input type="text" ng-model="velocidad_comprobada" placeholder="Ingrese velocidad comprobada">
    <h3>Velocidad máxima permitida {{velocidad_maxima}}</h3>
    <h3>Velocidad constatada {{velocidad_comprobada}}</h3>
    <hr>
    <h1>Rebasamiento de velocidad</h1>
    <h3>el vehículo circuulaba {{velocidad_comprobada - velocidad_maxima}} por sobre la velocidad permitida</h3>
  </div>
</body>
</html>
```

Una primera mini app en AngularJS

Son dos documentos, 05.html y todo.js. El primero incluye el segundo en la cabecera. Tiene un estilo, de manera que se ve gráficamente las posibilidades de cambiar interactivamente los formatos de visualizacion, sin tener que hacer llamadas al servidor.

en principio, entender que todo el js es un controlador dentro de la aplicación angular.

en Angular, las funciones se declaran como atributos de el objeto, en este caso todoList.

Primero un objeto/atributo todoList.todos, que declara un objeto al estilo antiguo de JS (como un json).

En realidad necesitamos un array asociativo.

Este objeto admite también métodos de array, como push. en la función addTodo, lo utilizamos de modo que añada los nuevos pares de clave valor, seteando el campo realizado (done), como false.

La segunda es una función que recupera los valores de el objeto todos, siempre que done sea true.

La segunda, previamente guarda en copia todos con otro nombre, sobre el que itera.

A cada valor pregunta si es true, y en este caso, vuelve a cargar la lista todos solamente con los valores por realizar

todo.js

```
angular.module('todoApp', [])
.controller('TodoListController', function() {
  var todoList = this;
  todoList.todos = [
    {text:'aprender AngularJS', done:true},
    {text:'construir una app en AngularJS', done:false}];

  todoList.addTodo = function() {
    todoList.todos.push({text:todoList.todoText, done:false});
    todoList.todoText = '';
  };

  todoList.remaining = function() {
    var count = 0;
    angular.forEach(todoList.todos, function(todo) {
      count += todo.done ? 0 : 1;
    });
    return count;
  };

  todoList.archive = function() {
    var oldTodos = todoList.todos;
    todoList.todos = [];
    angular.forEach(oldTodos, function(todo) {
      if (!todo.done) todoList.todos.push(todo);
    });
  };
});
```

html

Dentro del div general se declara como atributo ng-controller, con el nombre que se le creo en el js.

es en realidad en este momento en el que instanciamos el controlador. Y es por eso que el objeto que utilizará (todoList), se inicializa como this; un objeto de la clase del controlador, que será encargado de manejar el comportamiento de todo el contenido del div.

Luego llama a la función remaining que retorna la cantidad de tareas. Y luego en una etiqueta a, en vez de acceder a un enlace, accede a la función archive a través del atributo ng-click.

atención porque las llamadas a las funciones son regulares, cuando las declaraciones se hacen creando primero la variable con el nombre de la función, y luego atribuyéndole function(){código a ejecutar}

```
<div ng-controller="TodoListController as todoList">
  <span>{{todoList.remaining()}} de {{todoList.todos.length}} tareas por hacer</span>
  [ <a href="" ng-click="todoList.archive()">archivar las realizadas</a> ]
```

Veremos que en principio tenemos una funcionalidad de angular ng-repeat, que nos permite iterar sobre valores como arrays u objetos.

```
<li ng-repeat="todo in todoList.todos">
```

Luego dentro del campo de cada uno, a través de un estilo css, que se carga solamente en los casos en que el valor es true, mostraremos tachadas las tareas realizadas con el formato que ya hemos visto

```
<label class="checkbox">
  <input type="checkbox" ng-model="todo.done">
  <span class="done-{{todo.done}}">{{todo.text}}</span>
</label>
```

El mini-formulario, no utiliza action ni post, sino un atributo de angular ng-submit, que llama a una función de todo. En concreto la de añadir. posteriormente un botón type submit dispara esta función

```
<input type="text" ng-model="todoList.todoText" size="30"
  placeholder="añade tarea pendiente">
```

?

El código HTML

```
<!doctype html>

<html ng-app="todoApp">

  <head>

    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

    <script src="todo.js"></script>

    <style>

      .done=true {

        text-decoration: line-through;

        color: grey;

      }

    </style>

  </head>

  <body>

    <h2>Por hacer</h2>

    <div ng-controller="TodoListController as todoList">

      <span>{{todoList.remaining()}} de {{todoList.todos.length}} tareas por hacer</span>

      [ <a href="" ng-click="todoList.archive()">archivar las realizadas</a> ]

      <ul class="unstyled">

        <!-- iteramos sobre las tareas pendientes -->

        <li ng-repeat="todo in todoList.todos">

          <label class="checkbox">

            <input type="checkbox" ng-model="todo.done">

            <span class="done-{{todo.done}}">{{todo.text}}</span>

          </label>

        </li>

      </ul>

      <form ng-submit="todoList.addTodo()">

        <input type="text" ng-model="todoList.todoText" size="30"

          placeholder="añade tarea pendiente">

        <input class="btn-primary" type="submit" value="add">

      </form>

    </div>

  </body>

</html>
```

